

SERVICE ACTUEL · IAM

Identités métier, autorisations et administration

8 capacités analysées

services/iam · Python, FastAPI et SQLAlchemy asynchrone ; PostgreSQL, OpenFGA, APISIX, Keycloak et Kafka dans les intégrations actuelles.

Gérer les personnes, groupes, projets, comptes techniques et décisions d'accès, puis projeter ces droits dans OpenFGA et APISIX. Les utilisateurs disposent de leur profil, sessions et partages ; les administrateurs pilotent les identités et la sécurité.

Lecture de code**Pas de qualification runtime nouvelle**

Domaines cible : IAM, Audit, Catalog, Query, Files, MCP, Applications.

Périmètre de lecture

- AGENTS.md du composant ; rapport identité-gouvernance existant.
- Routes utilisateurs, organisations, équipes, membres, projets, partages, brokers, sessions, tokens et administration.
- Handlers de partage et transfert, services de tokens et comptes techniques, délégation, APISIX et outbox. Lecture ciblée, pas audit intégral de chaque branche.

Objets et cycle de vie

- User garde identité métier, sujet de connexion, état et statut administrateur ; le sujet JWT ne doit pas être confondu avec la clé interne.
- Organisations, équipes, memberships, projets et espaces personnels portent la hiérarchie et les rôles.
- ServiceAccount et ExpositionApiToken décrivent des identités et credentials d'intégration, pas des éditions de données.
- Routes APISIX, audits, état break-glass et outbox ont une autorité PostgreSQL ; les tuples sont des projections et relations de décision.

Dépendances du service

- Keycloak assure actuellement connexion, clients, sessions et fédérations.
- OpenFGA évalue les relations ; APISIX applique le routage et les contrôles gateway.
- Kafka distribue les faits aux miroirs ; Kubernetes reçoit certains secrets et objets d'intégration.

Fonctionnalités et compatibilité à préserver

IAM-01 Cible non qualifiée

Administrer les personnes et organiser leur départ

COMPORTEMENT OBSERVÉ

Les routes couvrent profil courant, langue, annuaire, création, mise à jour, désactivation et suppression. Le drapeau local `is_platform_admin` est l'autorité métier pour les administrateurs de plateforme ; `set_platform_admin` écrit aussi projection de droit, audit et événement. Retirer le dernier administrateur actif est refusé. L'offboarding désigne des héritiers par famille de ressources puis désactive ou supprime la personne, avec un événement `iam.user.offboarded` ordonné avant le fait utilisateur courant dans la même transaction. Une simple suppression d'utilisateur n'a pas ces sémantiques de réaffectation.

UTILISATEURS ET RÔLES

Utilisateur pour son profil ; administrateur pour le cycle de vie et le départ.

INTERFACES DANS LE CODE

GET/PUT `/api/v1/users/me` ; PUT `/api/v1/users/me/lang` ; CRUD `/api/v1/users` ; POST `/api/v1/admin/users/{user_id}/offboard` ; `UserService.set_platform_admin`.

DONNÉES ET ÉTATS

User, statut actif/supprimé, identifiant métier et sujet Keycloak, drapeau administrateur, audit et événements outbox.

DÉPENDANCES ET ÉCHANGES

Keycloak pour identité de connexion ; services propriétaires pour réaffectation ; OpenFGA et Kafka pour projections.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust conserve une identité métier stable et un mapping explicite `issuer/sub`. Le départ devient une commande idempotente avec héritiers validés et suivi des accusés des services propriétaires. Le dernier administrateur reste protégé sous concurrence.

MIGRATION DE CETTE CAPACITÉ

Importer IDs, sujets, statuts et correspondances sans fusion par seul courriel. Porter les codes d'erreur et publier les mêmes faits via un transport sans JVM. Faire converger les réaffectations avant retrait des anciens consommateurs.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux retraits concurrents ne peuvent supprimer le dernier administrateur. Rejouer une commande d'offboarding ne transfère pas deux fois les ressources et conserve le même événement métier.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/user_service.py:3587](#) Écriture centralisée du statut administrateur et garde du dernier actif.

[monorepo · services/iam/src/iam_service/api/v1/admin/admin_offboarding.py:75](#) Contrat de départ avec réaffectations et ordre des événements.

[monorepo · services/iam/src/iam_service/api/v1/users.py:322](#) Édition de langue du profil courant.

IAM-02 Cible non qualifiée

Structurer organisations, équipes, projets et memberships

COMPORTEMENT OBSERVÉ

Les API gèrent organisations, équipes et appartenances avec rôles, ainsi que projets, membres et espaces personnels. Les projets gardent un ID interne et des alias hérités. Le transfert de propriétaire prend le verrou du projet, modifie memberships, audit et outbox dans la transaction, puis utilise une écriture OpenFGA contrôlée sur les alias. Le propriétaire sortant devient manager par défaut ; le départ peut retirer son membership. PostgreSQL et OpenFGA n'ont pas de transaction distribuée commune.

UTILISATEURS ET RÔLES

Propriétaires, responsables de projets et équipes, administrateurs.

INTERFACES DANS LE CODE

CRUD `/api/v1/organizations`, `/api/v1/teams`, `/api/v1/projects` ; `/api/v1/memberships` ;

`ProjectService.transfer_owner` ; `OrganizationService.transfer_ownership`.

DONNÉES ET ÉTATS

Organization, Team, Membership, Project et ProjectMember ; correspondances `legacy_id` ; relations OpenFGA.

DÉPENDANCES ET ÉCHANGES

OpenFGA, Keycloak pour les sujets, miroirs utilisateurs/projets des autres services.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust porte les invariants de hiérarchie, les espaces personnels et transferts. Une intention durable identifie les mutations externes et permet de réconcilier les tuples sans transformer un échec de lecture en graphe vide.

MIGRATION DE CETTE CAPACITÉ

Importer la hiérarchie et tous les alias avant de changer le routage. Comparer les relations effectives et les listings pour des profils réels ; garder un seul auteur des memberships par cohorte.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Deux transferts concurrents se sérialisent. Une panne du graphe ne laisse pas un ancien propriétaire rétabli par le prochain reconciler. L'espace personnel ne devient pas partageable par accident.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/project_service.py:1361](#) Verrouillage, propriétaire sortant et limite transactionnelle.

[monorepo · services/iam/src/iam_service/services/organization_service.py:633](#) Transfert au niveau organisation.

[monorepo · services/iam/src/iam_service/api/v1/memberships.py:166](#) Contrat des rôles de membership.

IAM-03 Cible non qualifiée

Décider les droits de lecture et d'action à partir de faits contrôlés

COMPORTEMENT OBSERVÉ

Les brokers internes authentifient le client appelant avec JWT vérifié, émetteur, audience configurée et azp attendu. Ils résolvent un utilisateur actif puis demandent une décision OpenFGA avec cohérence renforcée. Le broker dataset accepte de 1 à 100 contrôles et au plus 12 projets liés par contrôle. Il distingue consultation, téléchargement, accès projet et token. Un compte technique ne télécharge jamais un dataset RESTRICTED ; un rôle AGENT stocké ne contourne pas cette règle. Une décision indisponible provoque 503, sans tableau partiellement autorisé.

UTILISATEURS ET RÔLES

Clients de service de confiance agissant pour une personne ou un compte technique.

INTERFACES DANS LE CODE

DatasetAuthorizationRequest ; POST /data-serve/authorize dans le routeur dataset_authorization ; decision_broker.verify_broker_identity et evaluate_subject_checks ; autres brokers typés files, ingestion, development, application, model et runner.

DONNÉES ET ÉTATS

Utilisateur actif et comptes techniques PostgreSQL ; relations OpenFGA ; faits contextuels de requête non persistés.

DÉPENDANCES ET ÉCHANGES

OpenFGA ; identité OIDC ; faits résolus par le service propriétaire, sans rôles arbitraires fournis par le client.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust expose un contrat de décision typé liant sujet, acteur, tenant, audience, opération, ressource ou édition et version de politique. Query et Files restent les points de médiation des lignes, colonnes et octets ; un droit Lakekeeper table.modify ne devient pas un droit générique de worker.

MIGRATION DE CETTE CAPACITÉ

Porter les matrices de décision et les routes sans assouplir les refus. Comparer les décisions ancienne/nouvelle en mode observation avant bascule, avec invalidation bornée et cas de panne.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un mauvais azp, un utilisateur inactif et un export RESTRICTED par compte technique sont refusés. Une panne sur la dernière décision d'un lot n'autorise pas les premières par une réponse partielle.

Références de l'analyse

[monorepo · services/iam/src/iam_service/api/v1/decision_broker.py:18](#) Identité des brokers et vérification JWT obligatoire.

[monorepo · services/iam/src/iam_service/api/v1/dataset_authorization.py:32](#) Champs et limites du contrat.

[monorepo · services/iam/src/iam_service/api/v1/dataset_authorization.py:123](#) Règles humaines/techniques, états actifs et refus en cas de panne.

IAM-04 **Cible non qualifiée****Partager une ressource et remplacer réellement le rôle accordé****COMPORTEMENT OBSERVÉ**

Les routes partagent applications, fichiers et dossiers avec un utilisateur ou une équipe, révoquent le partage, listent les permissions et les ressources partagées avec soi. SharingService traite aussi les projets. Un partage fixe un rôle ; il ne cumule pas un nouveau rôle avec un rôle plus fort déjà détenu. Le service recense les rôles, accorde le rôle voulu puis retire ceux remplacés sur les alias. Une personne sans sujet d'identité exploitable est refusée avant le premier tuple. Les restrictions de propriétaire et de représentation des memberships restent distinctes d'un partage simple.

UTILISATEURS ET RÔLES

Propriétaire ou gestionnaire de partages ; bénéficiaire utilisateur ou équipe.

INTERFACES DANS LE CODE

POST/DELETE /api/v1/sharing/{applications|files|folders}/{id}/share ; GET permissions ; GET /api/v1/sharing/shared-with-me ; SharingService.share_resource.

DONNÉES ET ÉTATS

Tuples utilisateur ou équipe, aliases de ressource, memberships projet et audit.

DÉPENDANCES ET ÉCHANGES

OpenFGA et résolution des sujets ; miroir des memberships projet.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust garde une commande set-role explicite et une commande revoke. Les résultats portent l'état effectif et la version de décision. Catalog collections et EXPOSURE réutilisent ces identités sans conférer implicitement des droits sur les données.

MIGRATION DE CETTE CAPACITÉ

Recenser droits directs, hérités et alias. Rejouer promotions et rétrogradations sur une copie du graphe, puis migrer les écritures avec reprise des opérations incomplètes.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Passer de admin à viewer retire le droit admin sur tous les alias. Une indisponibilité du recensement ne peut répondre succès. Un partage de collection ne permet pas de lire un dataset interdit.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/sharing_service.py:261](#) Sémantique set-role, recensement, grant puis revoke.

[monorepo · services/iam/src/iam_service/api/v1/sharing.py:326](#) Liste des ressources partagées au sujet.

IAM-05 **Cible non qualifiée****Administrer comptes techniques, tokens d'exposition et délégations MCP****COMPORTEMENT OBSERVÉ**

IAM découvre, enregistre, met à jour, synchronise et retire des comptes techniques avec portée de projet ou organisation. Les tokens d'exposition ont propriétaire, périmètre, routes et date d'expiration. Créer un token déjà expiré conserve la ligne mais ne provisionne pas de consommateur actif dans APISIX ; modifier sa date conserve sa clé, et passer dans le passé déclenche la révocation. Le reconciler de délégation configure l'échange standard uniquement pour les clients prévus ; la délégation de l'assistant reste dans la session utilisateur. Les outils dataset/corpus passent par read-identity avec leurs comptes dédiés.

UTILISATEURS ET RÔLES

Administrateurs, propriétaires de projets, intégrations et assistant délégué.

INTERFACES DANS LE CODE

ServiceAccountService.register/update/unregister/sync ; ExpositionTokenService.create_token/patch_token ; exposition_tokens router ; POST /mcp/read-identity dans le routeur dédié ; reconcile_keycloak_delegation.

DONNÉES ET ÉTATS

ServiceAccount, User technique, ExpositionApiToken, consommateurs/routes APISIX et configuration de clients OIDC.

DÉPENDANCES ET ÉCHANGES

Keycloak, APISIX, Kubernetes Secrets et broker IAM de résolution d'acteur.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust distingue personne, acteur et identité de service. Les capacités courtes sont liées au run et à une opération précise. Les tokens d'accès aux API restent des credentials ; EXPOSURE devient séparément un consommateur déclaré d'éditeurs.

MIGRATION DE CETTE CAPACITÉ

Importer portées et propriétaires, renouveler les secrets au changement d'IdP et mesurer la révocation. Ne pas présumer portables les sessions ni refresh tokens. Préserver les intégrations par une façade d'authentification bornée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un token créé expiré ne donne jamais accès. Fermer la session utilisateur invalide la délégation associée. Un compte limité à un projet ne gagne aucun accès par substitution de X-User-Id.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/service_account_service.py:206](#) Enregistrement et portée des comptes techniques.

[monorepo · services/iam/src/iam_service/services/exposition_token_service.py:400](#) Traitement des credentials déjà expirés et provisioning APISIX.

[monorepo · services/iam/src/iam_service/services/keycloak_delegation.py:2](#) Contrat de délégation et distinction des outils dataset/corpus.

IAM-06 **Cible non qualifiée****Gérer sessions, fédérations et politiques de connexion****COMPORTEMENT OBSERVÉ**

Une personne liste ses sessions et révoque une session ou les autres sessions ; le sujet vient de l'identité authentifiée. Les API d'administration lisent, modifient, réinitialisent et audient politiques de mots de passe et sessions. La fédération est stockée dans la représentation d'IdP Keycloak, avec extensions de configuration ; le mode de sécurité global est en base IAM. Passer en SSO_ONLY exige un fournisseur actif. Supprimer ou désactiver le dernier fournisseur rétablit le mode qui permet les credentials, pour éviter un blocage du login.

UTILISATEURS ET RÔLES

Utilisateurs pour leurs sessions ; administrateurs pour fédérations et politiques.

INTERFACES DANS LE CODE

GET/DELETE /api/v1/users/me/sessions ; DELETE /api/v1/users/me/sessions/{session_id} ; /api/v1/admin/identity-providers ; admin_security_policies router.

DONNÉES ET ÉTATS

Sessions et représentations de fournisseurs dans Keycloak ; system_config et audits de politique dans PostgreSQL.

DÉPENDANCES ET ÉCHANGES

API Admin Keycloak, system_config, audit et frontend de connexion.

COMPORTEMENT CIBLE PROPOSÉ

IAM Rust conserve les décisions métier et délègue l'authentification à un fournisseur sans JVM qualifié. Les politiques et fédérations utilisent un modèle versionné avec audit ; la suppression du dernier moyen de connexion est refusée ou compensée explicitement.

MIGRATION DE CETTE CAPACITÉ

Inventorier fédérations, paramètres de mots de passe, MFA et lifespans sans exporter de secrets dans le dossier. Tester le transfert des credentials ; réenrôler les moyens non portables et imposer une réauthentification lors de la coupure.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Un utilisateur ne peut révoquer la session d'un autre. Retirer le dernier IdP ne laisse pas une page SSO sans voie de connexion. Une révocation arrête aussi les accès délégués dans la fenêtre contractuelle.

Références de l'analyse

[monorepo · services/iam/src/iam_service/api/v1/user_sessions.py:283](#) Révocation des sessions du sujet courant.

[monorepo · services/iam/src/iam_service/api/v1/admin/admin_identity_providers.py:23](#) Autorité de stockage et garde anti-blocage.

[monorepo · services/iam/src/iam_service/services/security_policy_service.py:322](#) Écriture des politiques de session et audit.

IAM-07 **Cible non qualifiée****Réconcilier le routage et les barrières d'accès de la gateway****COMPOTEMENT OBSERVÉ**

IAM possède la table de routes APISIX de la plateforme. Au démarrage, `ensure_seed_data` rapproche le seed des lignes stockées, puis les synchronisations rendent routes, upstreams et plugins. Le service propose synchronisation et diff. Les types `oidc` et `oidc_openfga` séparent authentification et décision ; une route `oidc_openfga` sans configuration de droit est refusée. Des règles protègent les routes gérées par les instances et empêchent les rétrogradations d'authentification non permises. Retirer des lignes du seed entraîne un nettoyage ciblé des routes obsolètes.

UTILISATEURS ET RÔLES

Exploitation, administrateurs et services créateurs de routes d'instances.

INTERFACES DANS LE CODE

`authz/seed_routes.py` ; `ApisixService.ensure_seed_data/sync_all/diff` ; `admin_routes` et `instance_routes`.

DONNÉES ET ÉTATS

`ApisixRoute`, `Upstream`, `PluginConfig` et `Consumer PostgreSQL` ; rendu externe dans APISIX.

DÉPENDANCES ET ÉCHANGES

APISIX Admin, OIDC, OpenFGA et services amont.

COMPOTEMENT CIBLE PROPOSÉ

IAM Rust compile une politique de routage versionnée avec inventaire des exceptions publiques. Les identités et décisions sont propagées aux services sans permettre à un header utilisateur non vérifié de remplacer le sujet.

MIGRATION DE CETTE CAPACITÉ

Exporter le seed et comparer le rendu réel, y compris routes plates, WebSocket et endpoints publics. Basculer les routes par famille ; conserver un diff et une procédure de retour qui ne réactive pas une route retirée.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une route privée ajoutée sans politique est refusée avant déploiement. Une route retirée du seed disparaît sans supprimer les routes d'instances encore actives. Une tentative de passage à `auth_type none` est bloquée.

Références de l'analyse

[monorepo · services/iam/src/iam_service/services/apisix_service.py:1655](#) Rapprochement du seed et nettoyage associé.

[monorepo · services/iam/src/iam_service/services/apisix_service.py:1291](#) Validation de la configuration de droit.

[monorepo](#) · [services/iam/src/iam_service/api/v1/admin/admin_routes.py:62](#) Protection des objets gérés par les instances.

IAM-08 **Cible non qualifiée**

Auditer, réconcilier les droits et ouvrir un accès de secours borné

COMPORTEMENT OBSERVÉ

Les mutations produisent audits et faits relayés par une outbox. Le reconciler compare projections et relations, avec un traitement particulier des administrateurs de plateforme. Le break-glass s'active depuis une identité déjà administrateur actif en base locale, sans créer de sujet inconnu pendant la panne. Il a une portée et une expiration ; son rendu assouplit certaines décisions gateway tout en conservant l'authentification. Les écritures IAM et routes avec contrôle system:global restent protégées. La sortie manuelle ou l'expiration déclenche une réconciliation lorsque le service requis est disponible.

UTILISATEURS ET RÔLES

Administrateurs de plateforme, exploitation et auditeurs.

INTERFACES DANS LE CODE

/api/v1/admin/audit ; admin_breakglass router ; BreakGlassService.set_break_glass/expire_if_due ; OpenFGAReconciler ; EventOutboxDispatcher.

DONNÉES ET ÉTATS

AuditLog, AuthzSystemState, EventOutbox et projections OpenFGA.

DÉPENDANCES ET ÉCHANGES

PostgreSQL, OpenFGA, APISIX et transport actuel Kafka.

COMPORTEMENT CIBLE PROPOSÉ

IAM et Audit Rust gardent l'intention, le motif, la portée, l'échéance et les preuves d'application. La lecture d'une politique dégradée reste explicite. Le transport sans JVM conserve ordre par ressource et reprise, sans transformer l'audit IAM en preuve de publication de données.

MIGRATION DE CETTE CAPACITÉ

Importer historique utile et état de secours, puis qualifier panne et restauration. Remplacer Kafka avec maintien des clés et identifiants d'événement. Garder les mécanismes de rattrapage jusqu'à convergence des projections.

CRITÈRE D'ACCEPTATION À EXÉCUTER

Une activation expirée n'assouplit plus aucune route même si la boucle de nettoyage tarde. Une personne inactive ne peut activer le secours. Après panne puis reprise, l'audit et le graphe concordent avec les mutations durables.

Références de l'analyse

[monorepo](#) · [services/iam/src/iam_service/api/v1/admin/admin_breakglass.py:54](#) Autorité locale et refus de provisionner un sujet inconnu.

[monorepo](#) · [services/iam/src/iam_service/services/breakglass_service.py:367](#) Expiration de l'activation.

[monorepo](#) · [services/iam/src/iam_service/services/event_outbox_dispatcher.py:312](#) Prise en charge durable des événements à relayer.

Architecture et exploitation du service cible

Plan de contrôle Rust proposé

- Identités, relations, délégations, politiques, sessions métier, audit des mutations et réconciliation versionnée.
- Décisions sur ressources et éditions avec sujet et acteur distincts ; autorité des commandes en PostgreSQL.

Plan de données proposé

- IAM ne transporte pas les lignes ni les blobs. Query, Files et Writer consomment des décisions liées à un périmètre précis.
- Le Publisher contrôle séparément la publication ; les workers ne reçoivent pas de permission REST générique table.modify.

Découpage du core Rust

- Modules proposés identity, membership, policy, delegation, route-policy et reconciliation, avec adaptateurs OIDC/OpenFGA/APISIX.
- Outbox Rust transactionnelle et interfaces de provisioning idempotentes pour les dépendances externes.

Identité, autorisation et isolation

- Préserver toutes les restrictions propres aux comptes techniques, la garde du dernier administrateur et les décisions indisponibles.
- Mesurer révocation et cache jusqu'aux flux ouverts ; ne pas supposer que la durée d'un JWT suffit à prouver une révocation.

Exploitation et reprise

- Sauvegarder PostgreSQL et les représentations d'identité nécessaires à la reprise ; restaurer puis réconcilier avec OpenFGA et gateway.
- Superviser le retard d'outbox, les décisions dégradées, les divergences et l'échéance de secours. La cible retire Keycloak et Kafka, y compris leurs tâches d'exploitation JVM.

Séquence de migration

- Figurer les contrats HTTP et les matrices de décision à partir des tests existants.
- Importer les IDs et mappings issuer/sub ; comparer sans écrire les droits rendus par Rust.
- Basculer les mutations par famille vers un auteur unique ; qualifier départ, révocation, panne et restauration.

- Retirer les composants JVM lorsque les parcours de connexion et délégation ont leur preuve de remplacement.

Écarts et décisions encore ouverts

- Le fournisseur OIDC sans JVM reste à qualifier sur les fédérations et moyens MFA réellement utilisés. Le rapport original cite un candidat, sans preuve d'intégration.
- Cette lecture ciblée ne couvre pas chaque branche des nombreux contrôleurs administratifs. Les contrats détaillés des ressources spécialisées doivent conserver leurs tests existants.
- Aucun test, appel à un IdP ou scénario de panne n'a été exécuté pour ce lot.

Références de l'analyse

[monorepo](#) · [services/iam/src/iam_service/main.py:2542](#) Assemblage des familles de routes.

[monorepo](#) · [services/iam/src/iam_service/services/apisix_service.py:1942](#) Démarrage de la configuration gateway.